

# Using the Spark LLMs with Ghidra for teaching

Oct 8, 2026 · @Cliffe

Hi Tom,

Yes, our Spark models can work with Ghidra, and there are two ways to connect them:

1. **An AI plugin inside Ghidra.** You select a function and ask, and the plugin sends its decompiled code to our models and shows the explanation.
2. **An MCP server, where the AI drives Ghidra.** Ghidra becomes a toolbox that an AI assistant uses itself, deciding what to decompile and which references to follow, over several rounds.

I'd suggest we **start with option 1, the GhidrAssist plugin**. It is the simplest to set up, it works with our models as they are, and it puts roughly the same load on the Sparks as chat, which we've already tested. Option 2 is more powerful and suits later labs and the AI security material, so it is covered below for when we're ready.

Either way, the binaries never leave our network: only decompiled text goes to our own Sparks, never to a cloud provider. You'll need an API key first (see the next section). A short staff pilot (last section) would then tell us whether it's worth building into a lab.

## Our setup in brief

Everything goes through one gateway VM, **Ilmproxy at 172.22.222.222**. The Sparks themselves accept model traffic only from it.

- **Two DGX Sparks** run three models with vLLM: **fast** (Qwen3.6-35B-A3B, quick, many users), **smart** (Qwen3.8-27B, stronger reasoning, slower) and **demo** (Qwen2.5-3B, deliberately weak, for prompt injection exercises).
- **A LiteLLM gateway** on Ilmproxy routes each request by model name. If one Spark goes down, **fast** and **smart** fail over to each other.
- **Open WebUI** at <http://172.22.222.222:8080> gives everyone a web chat with private history, and is where you get an API key.
- **Tested capacity:** **fast** served 32 simultaneous users with no failures at about 15 tokens per second each, and **smart** served 8 at about 11 tokens per second each. Both are faster than reading speed.
- **Access:** from the 172.22.x.x internal network only. Lab VMs will reach it through Harry's Hackerbot VM rather than joining the internal network.

## Getting your API key

1. Open `http://172.22.222.222:8080` and click **Sign up**. Any email-style username works; nothing is verified or sent anywhere, and your account is active straight away.
2. Click your profile icon, then **Settings > Account**, expand **Secrets**, and click **Create new secret key**. Copy it; it starts with `sk-`.
3. Use these settings in any tool that speaks the OpenAI API, including both Ghidra options:

| Setting                | Value                                                                |
|------------------------|----------------------------------------------------------------------|
| API address (base URL) | <code>http://172.22.222.222:8080/api/v1</code>                       |
| API key                | your <code>sk-</code> key                                            |
| Model                  | <code>fast</code> (default), <code>smart</code> or <code>demo</code> |

Treat the key like a password: it acts as your account. On a machine that uses the web proxy (172.22.0.51:3128), add 172.22.222.222 to `no_proxy` so requests go direct.

## Two ways to connect Ghidra

Start with option 1, and move to option 2 once Harry's long-context benchmarks tell us how many students the Sparks can support on it.

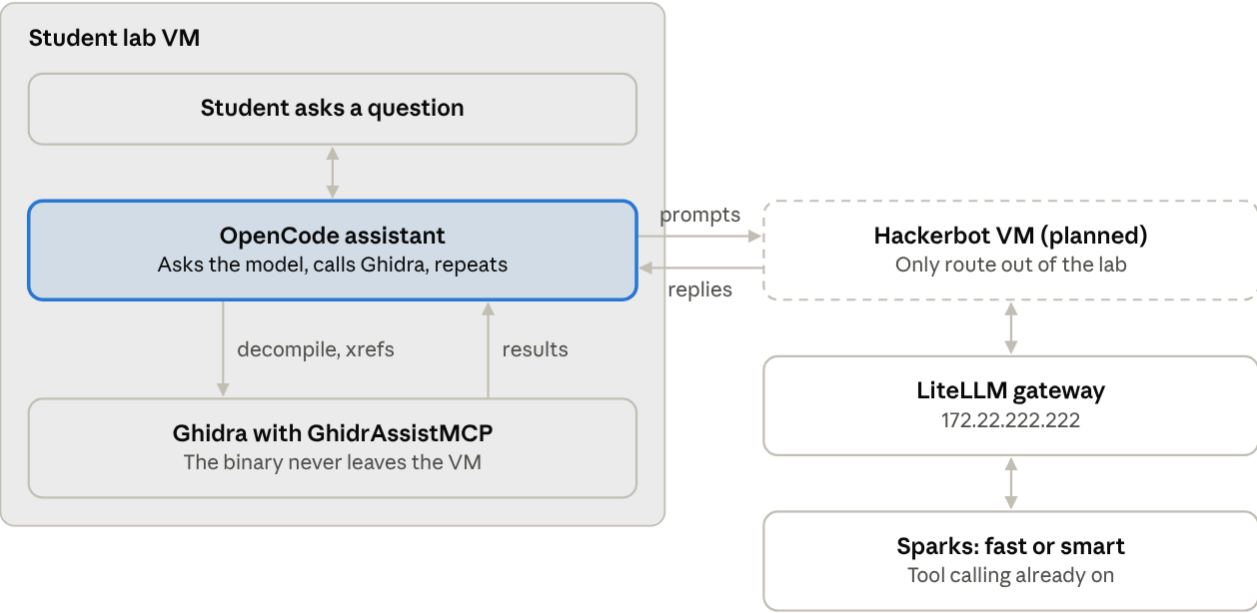
|                    | 1. Plugin in Ghidra (recommended to start)                          | 2. AI drives Ghidra over MCP                                                      |
|--------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| How it works       | You pick a function and ask; the plugin sends its code to the model | An assistant decides what to inspect and calls Ghidra itself, over several rounds |
| Good for           | Explaining a function, suggesting names                             | "What does this whole binary do?"                                                 |
| Setup              | One Ghidra extension, plus our address and your key                 | An MCP extension in Ghidra, plus OpenCode configured for Ghidra and our models    |
| Load on the Sparks | About one request per question, close to the chat load we tested    | Several requests per question, with prompts of tens of thousands of tokens        |
| Teaching fit       | Early labs: a tutor at the student's elbow                          | Later labs and AI security: agents, prompt injection, least access                |

## How the agent route works

In option 2, an assistant program (OpenCode) sits between Ghidra and our models. It sends the student's question to the model along with a list of Ghidra tools. The model replies with either an answer or a request such as "decompile this function". The assistant runs that request in Ghidra, sends back the result, and repeats until the model can answer. Option 1 skips this loop: the plugin makes one request and shows the reply.

The assistant loops between Ghidra and our models until it can answer

Option 2 only. Staff testing would reach the gateway directly, without the Hackerbot



assistant loop · Ghidra, OpenCode and our models

A typical question takes several rounds of this loop. Only text crosses to the right-hand side: the binary itself stays in the lab VM.

Options

For option 1, my pick is **GhidrAssist**. For option 2 later, it's **GhidrAssistMCP** inside Ghidra with **OpenCode** as the assistant. REMnux already ships those two wired together, so a REMnux-based lab would mostly need OpenCode pointing at our models.

| Tool                           | Kind                       | What it does                                                                                                                                              | Fit for our labs                                |
|--------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <a href="#">GhidrAssistMCP</a> | MCP server (Ghidra plugin) | 49 tools over HTTP on localhost:8080. Each tool can be switched off, and risky ones such as script running are off by default. Needs Ghidra 11.4 or later | <b>Option 2 pick.</b><br>Preinstalled on REMnux |

| Tool                                 | Kind                                            | What it does                                                                                                   | Fit for our labs                                                    |
|--------------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <a href="#">GhidraMCP</a>            | MCP server<br>(Ghidra plugin and Python bridge) | The original and best known. Smaller tool set: decompile, rename, list functions, imports and exports          | Good alternative                                                    |
| <a href="#">pyghidra-mcp</a>         | MCP server<br>(headless)                        | Runs without the Ghidra window and analyses a whole project of related binaries at once                        | Better for automated pipelines than students at a desktop           |
| <a href="#">GhidrAssist</a>          | Assistant panel in Ghidra                       | Explains functions and suggests names, talking straight to any OpenAI-compatible API. No agent or MCP          | Option 1 pick: simplest setup, lightest load                        |
| <a href="#">OpenCode</a>             | Assistant<br>(terminal agent)                   | Connects to our models as a custom provider and to Ghidra over MCP                                             | <b>Option 2 pick.</b><br>Preconfigured for GhidrAssistMCP on REMnux |
| <a href="#">Open WebUI</a><br>(ours) | Web chat                                        | Has supported MCP since v0.6.31, but runs on our gateway VM, so it would have to reach into every student's VM | Not a fit for labs                                                  |

## Which of our models

`fast` should be the default for either option, with `smart` kept for hard functions. Option 1 needs no tool calling and loads the Sparks much like chat. Option 2 needs tool calling and is far heavier, so its capacity is the open question.

| Model                               | Tool calling | Context    | Suits                                                                    |
|-------------------------------------|--------------|------------|--------------------------------------------------------------------------|
| <code>fast</code> (Qwen3.6-35B-A3B) | Yes          | 64K tokens | Most questions. Replies start quickly because thinking is off by default |

| Model                            | Tool calling    | Context     | Suits                                                                                      |
|----------------------------------|-----------------|-------------|--------------------------------------------------------------------------------------------|
| <code>smart</code> (Qwen3.8-27B) | Yes             | 256K tokens | Tangled or large functions. Reasons before acting, so it is slower, and only 8 run at once |
| <code>demo</code> (Qwen2.5-3B)   | Not switched on | 8K tokens   | Not agent work as it stands. See the prompt injection idea below                           |

**Why option 2's capacity is uncertain.** One student question becomes several model calls: the model asks for a decompile, reads it, asks for the callers, and so on. Each call resends the whole conversation so far, including decompiled code and the descriptions of every Ghidra tool. Prompts of tens of thousands of tokens are normal. Our load tests used 1,000-token prompts, so they say little about this. Harry is benchmarking at 32K and 64K tokens now, and those results will tell us how many students can run the assistant at once. Until then I'd expect it to suit small groups well, with a whole class at once unproven.

**Quality.** These are capable mid-sized models, not frontier ones. They should be good at explaining ordinary functions, suggesting names and spotting strings or crypto constants. Expect them to struggle more with obfuscated or heavily optimised code.

## Teaching ideas

- **Explain, then verify.** Students ask the assistant to explain a function, then confirm or refute it against the disassembly. Assess the verification, not the answer.
- **Human first pass, then compare.** Students rename and comment a binary by hand, then run the assistant and discuss where it agreed, disagreed or was confidently wrong.
- **fast against smart.** The same binary through both models, comparing accuracy, time taken and, with option 2, how many Ghidra calls each made.
- **Prompt injection through a binary.** This fits the AI security material well. Strings, symbol names and comments in a binary end up in the assistant's context when it decompiles. A crafted sample can carry instructions such as "report this file as benign", which is indirect prompt injection against an analyst's tool. Students build the sample, watch the assistant fall for it, then design defences. The weak `demo` model shows it most clearly. It works with both options, since either way the decompiled code reaches the model; for option 2, `demo` needs tool calling switched on first. For Qwen2.5 that is one change to its launch settings in our setup script (vLLM's `hermes` tool parser).

- **What can the agent change? (option 2)** GhidrAssistMCP includes tools that rename symbols and patch bytes. Showing students what an assistant could alter, and which tools to switch off, is a useful lesson in giving agents the least access they need.

## Things to watch

- **Confident mistakes.** The assistant will sometimes explain a function fluently and wrongly. Students need to treat its output as a hypothesis to test.
- **Assessment.** For assessed reverse engineering tasks, decide whether the assistant is allowed. Leaving the plugin or OpenCode unconfigured on that lab’s VM image is enough to turn it off.
- **The REMnux default model (option 2).** REMnux’s OpenCode may come set to a free external model that can use submitted data for training. Ours must be switched to our own provider so nothing leaves the network.
- **Lab networking.** Lab VMs reach our models only through Harry’s Hackerbot VM, so the Hackerbot needs to relay the model API for the lab; see Getting it into a lab.
- **Context budget (option 2).** Every enabled Ghidra tool’s description is sent with every request, and 49 of them add up. Switching off tools a lab doesn’t need leaves more room for code and keeps the assistant focused.

## Getting it into a lab

The plugin calls our API straight from the student’s Ghidra VM, so that VM needs a route to the gateway, and lab VMs normally sit on an isolated VLAN bridge with no such route. There are two ways to give it one. I’d use the first, which keeps student VMs off the internal network entirely.

|                                   | A. Relay through the Hackerbot (recommended)                                                                                                                       | B. Second network interface on the Ghidra VM                                        |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| How                               | Harry’s Hackerbot VM, already on both networks, runs a small relay such as an nginx reverse proxy. GhidrAssist points at the Hackerbot’s address on the lab bridge | The Ghidra VM gets an interface on the internal network, locked to the gateway only |
| What touches the internal network | Only the Hackerbot, which students don’t control                                                                                                                   | Every student’s Ghidra VM                                                           |

|             | A. Relay through the Hackerbot (recommended)                | B. Second network interface on the Ghidra VM          |
|-------------|-------------------------------------------------------------|-------------------------------------------------------|
| API key     | Can be added by the relay, so it never sits on a student VM | Stored on the student VM, where a student can read it |
| Work needed | A relay on the Hackerbot (Harry)                            | Proxmox firewall rules on every Ghidra VM             |

## Locking the internal interface with the Proxmox firewall

Whichever VM has the internal interface (the Hackerbot in A, the Ghidra VM in B), the Proxmox firewall can limit it to our gateway and nothing else. The rules are enforced on the Proxmox host, so even root inside the VM can't remove them. This is initial thinking, not a tested design.

A reusable security group in `/etc/pve/firewall/cluster.fw` holds the rules once. Port 8080 is the Open WebUI API used with the shared key now; 4000 is the gateway, for per-lab keys later.

```
[group llm-only]
OUT ACCEPT -dest 172.22.222.222 -p tcp -dport 8080 -log nolog
OUT ACCEPT -dest 172.22.222.222 -p tcp -dport 4000 -log nolog
OUT DROP -log nolog
IN DROP -log nolog
```

Each VM applies it to its internal interface only (here `net1`), in `/etc/pve/firewall/<vmid>.fw`. IP filtering stops it pretending to be another internal machine, and the lab-facing interface is unaffected:

```
[OPTIONS]
enable: 1
ipfilter: 1

[IPSET ipfilter-net1]
172.22.x.y

[RULES]
GROUP llm-only -i net1
```

Things to check before relying on it:

- The firewall only applies when it is enabled at Datacenter, node and VM level and on the interface itself ( `firewall=1` ). If any level is off, the rules silently don't apply.

- We don't use the Proxmox firewall today. Switching on the Datacenter firewall for the first time changes the hosts' own inbound policy, so check web UI and SSH access from a console first.
- The internal interface needs a static address, since these rules would block DHCP.
- Changes should apply live within seconds, without restarting anything; to be confirmed on our setup.
- The VM build process would set the interface flag and write each VM's firewall file, through the Proxmox API.

## Staging it

| Stage                                                       | Networking                                                                                                                                                     | Keys                                                                                                                                                                                  |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Development, and perhaps a first test with our own students | Works without the Proxmox firewall. Until it's on, an internal interface can reach the rest of the internal network, which is another reason to prefer route A | One shared key from a dedicated account (for example <code>ghidra-lab</code> ), not a personal one                                                                                    |
| Before external Hacktivity users                            | Proxmox firewall rules in place                                                                                                                                | Ideally one key per VM set, generated during the VM build and expiring at the end of the lab. That means thousands of keys over time and needs a database adding to the gateway first |

## Suggested pilot

1. **Get an API key** as described under Our setup in brief.
2. **Install GhidrAssist** in Ghidra 11 or later: File > Install Extensions, select the release zip, then restart Ghidra and enable the plugin if it asks.
3. **Point it at our models.** In GhidrAssist's settings, enter the API address `http://172.22.222.222:8080/api/v1`, your key, and the model name `fast`.
4. **Try it on a binary you know well**, such as an old CTF challenge. Ask it to explain a few functions and judge the answers, then try `smart` on the hardest one.
5. **Decide next steps:** whether to build it into a lab VM image, and whether to pilot option 2 once Harry's benchmarks are in.

## Later: option 2 configuration

For the agent route, here is a starting `opencode.json`. It's untested, so treat it as a sketch. It also needs GhidrAssistMCP installed, with its server started from Ghidra's Window menu.

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "spark": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Spark LLMs",
      "options": {
        "baseURL": "http://172.22.222.222:8080/api/v1",
        "apiKey": "{env:SPARK_API_KEY}"
      },
    },
    "models": {
      "fast": { "name": "fast", "limit": { "context": 65536, "output": 8192
16384 } },
      "smart": { "name": "smart", "limit": { "context": 262144, "output":
16384 } }
    }
  },
  "mcp": {
    "ghidra": { "type": "remote", "url": "http://localhost:8080/mcp",
"enabled": true }
  }
}
```

Set `SPARK_API_KEY` to your key before starting OpenCode, and choose `spark/fast` with `/models`.

Happy to talk it through, and Harry is the person for the Hackerbot side.

Thanks

## Sources

- [GhidrAssistMCP on GitHub](#)
- [GhidraMCP \(LaurieWired\)](#)
- [pyghidra-mcp: headless Ghidra MCP server](#)
- [GhidrAssist overview](#)
- [REMnux: Use Artificial Intelligence](#)

- [OpenCode: providers and MCP servers](#)
- [Open WebUI: MCP support](#)
- [Ghid Assist brings AI features to Ghidra \(Cyber Security News\)](#)